ELTE
FACULTY OF
INFORMATICS

PROGRAMMING

The Matrix and PoA's

Zsuzsa Pluhár
pluharzs@inf.elte.hu

Refreshing

Programming patterns

1. Summation
2. Counting
3. Maximum selection
4. Copy
5. Multiple item selection

problems with sum and "all"
↓
counting loop

6. Search
7. Decision
8. Selection

problem with "exists "
↓
conditional loop

Matrix

- **Array**: a set of data with the same "function" (~set/type) *in one dimension*
  - using *one index* for referencing an element: `x[i]`

	x
1	-4
2	2
3	5

- **Matrix**: a set of data with the same "function" (~set/type) *in two dimensions*
  - using two indexes for referencing an element: `x[i,j]`
  - Specification: `n∈N, m∈N, x∈Z[1..n,1..m]`
  - Algorithm: `x:Array[1..n,1..m:Integer]`
  - Code: `int[,] x = new int[n, m];`

x	1	2	3	4
1	-4	5	3	2
2	2	1	10	11
3	5	0	4	-5

Example - sudoku

Task: How many 5 did we write in the sudoku table?

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

counting

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Task: How many 5 did we write in the sudoku table?

Specification:

```
In: s∈N[1..9,1..9]
Out: cnt∈N
Pre: ∀i∈[1..9]:(∀j∈[1..9]:(0<=s[i,j]<=9)
Post: cnt=COUNT(i=1..9,j=1..9,s[i,j]=5)
Post: cnt=COUNT(i=??..??,s[??]=5)
```

we don't have such a shorting

1D data structure needed – > convert to an array

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Task: How many 5 did we write in the sudoku table?

Idea: represent the two-dimensional matrix in one dimension,
-> traversing the matrix in row-wise).

	1	2	3
1	1	2	3
2	4	5	6
3	7	8	9

1 2 3 4 5 6 7 8 9

```
i=(k-1) div 3 + 1
j=(k-1) mod 3 + 1
```

```
k=(i-1)*3+j
```

example: i=2; j=3 -> k=(2-1)*3+3=6

example: k=7 -> i=(7-1) div 3+1 =3; j=(7-1) mod 3 +1=1

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Task: How many 5 did we write in the sudoku table?

Specification:

```
In: s∈N[1..9,1..9]
Out: cnt∈N
Pre: ∀i∈[1..9]:(∀j∈[1..9]:(0<=s[i,j]<=9)
Post: cnt=COUNT(k=1..9*9,
                s[(k-1) div 9 +1,
                  (k-1) mod 9 +1]=5)
```

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Task: How many 5 did we write in the sudoku table?

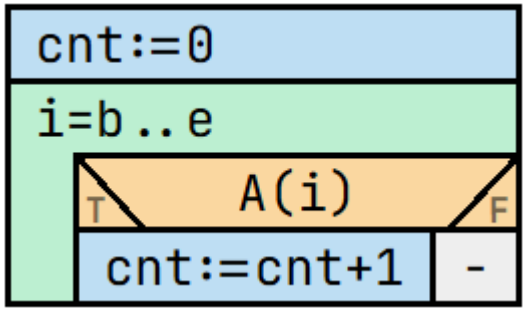
Pattern:

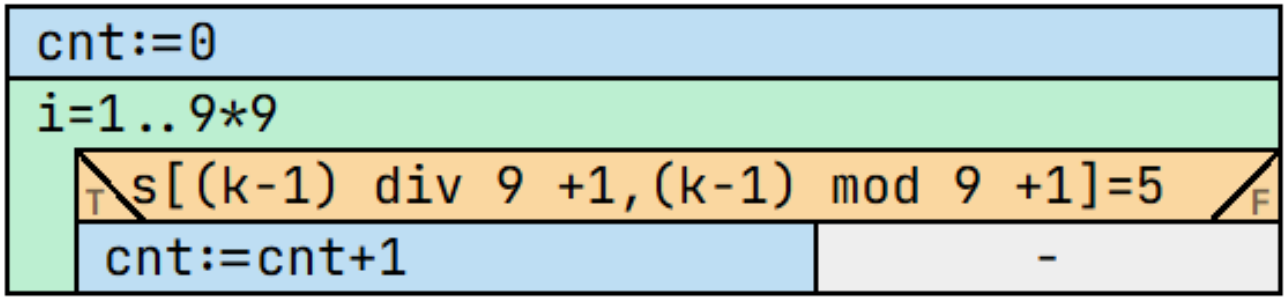
```
In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e,A(i))
```

Specification:

```
In: s∈N[1..9,1..9]
Out: cnt∈N
Pre: ∀i∈[1..9]:(∀j∈[1..9]:(0<=s[i,j]<=9)
Post: cnt=COUNT(k=1..9*9,
                s[(k-1) div 9 +1,
                  (k-1) mod 9 +1]=5)
```

i	~	k
b..e	~	1..9*9
A(i)	~	s[(k-1) div 9 + 1,(k-1) mod 9 + 1]=5





Example – sudoku indexing from 0

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Task: How many 5 did we write in the sudoku table?

Idea: represent the two-dimensional matrix in one dimension, -> traversing the matrix in row-wise).

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

1 2 3 4 5 6 7 8 9

```
i=k div 3
j=k mod 3
```

example: k=7 -> i=7 div 3 =2; j=7 mod 3=1

```
k=i*3+j
```

example: i=2; j=1 -> k=2*3+1=7

modify the specification

Code it!

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Task: How many 5 did we write in the sudoku table?

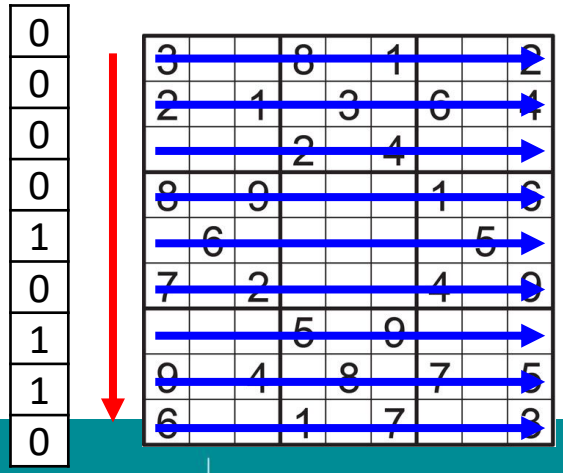
Idea:

Add up the number of fives in a row, row by row.

=> For each row, add up how many fives there are in a row.

summation

counting



a counting in a summation

cnt=3

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Task: How many 5 did we write in the sudoku table?

Specification:

```
In: s∈N[1..9,1..9]
Out: cnt∈N
Fn: cntfive: N->N, cntfive(i)= COUNT(j=1..9, s[i,j]=5))
Pre: ∀i∈[1..9]:(∀j∈[1..9]:(0<=s[i,j]<=9)
Post: cnt=SUM(i=1..9, cntfive(i))
```

s: global for the function!

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Summation pattern:

```
In: b∈Z, e∈Z
Out: s∈H
Pre: -
Post: s=SUM(i=b..e, f(i))
```

Counting pattern:

```
In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e, T(i))
```

Specification:

```
In: s∈N[1..9,1..9]
Out: cnt∈N
Fn: cntfive: N->N,
    cntfive(i)=COUNT(j=1..9, s[i,j]=5))
Pre: ∀i∈[1..9]:(∀j∈[1..9]:(0<=s[i,j]<=9)
Post: cnt=SUM(i=1..9, cntfive(i))
```

Summation		
s	~	cnt
b..e	~	1..9
f(i)	~	cntfive(i)

Counting		
i	~	j
b..e	~	1..9
A(i)	~	s[i,j]=5

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

cntfive(i)=cnt∈N: (cnt=COUNT(j=1..9,s[i,j]=5))

Summation pattern:

```
In: b∈Z, e∈Z
Out: s∈H
Pre: -
Post: s=SUM(i=b..e, f(i))
```

Counting pattern:

```
In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e, T(i))
```

Specification:

```
In: s∈N[1..9,1..9]
Out: cnt∈N
Fn: cntfive: N->N,
    cntfive(i)=COUNT(j=1..9, s[i,j]=5))
Pre: ∀i∈[1..9]:(∀j∈[1..9]:(0<=s[i,j]<=9)
Post: cnt=SUM(i=1..9, cntfive(i))
```

Summation		
s	~	cnt
b..e	~	1..9
f(i)	~	cntfive(i)

Counting		
i	~	j
b..e	~	1..9
A(i)	~	s[i,j]=5

Example - sudoku

3			8		1			2
2		1		3		6		4
			2		4			
8		9				1		6
	6						5	
7		2				4		9
			5		9			
9		4		8		7		5
6			1		7			3

Summation:

```
s:=0
i=b..e
  s:=s+f(i)
```

Summation

s	~	cnt
b..e	~	1..9
f(i)	~	cntfive(i)

```
s:=0
i=1..9
  s:=s+cntfive(i)
```

Counting:

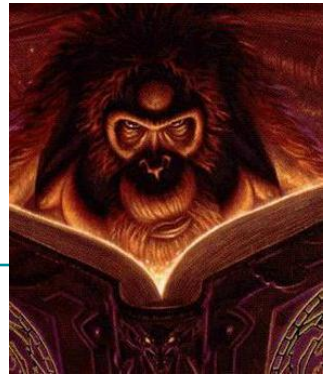
```
cnt:=0
i=b..e
  A(i)  T / F
  cnt:=cnt+1 | -
```

Counting

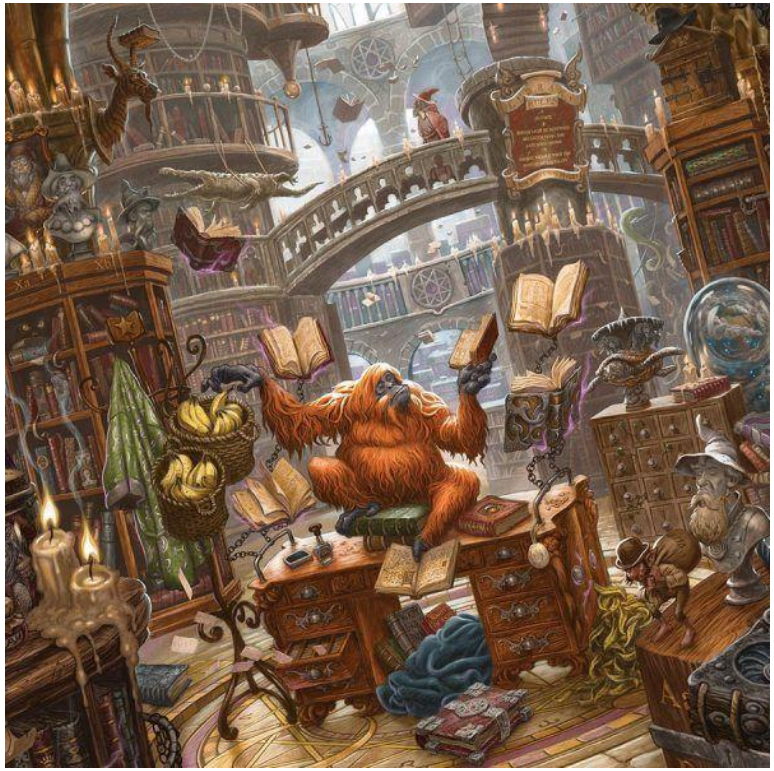
i	~	j
b..e	~	1..9
A(i)	~	s[i,j]=5

```
Function
cntfive(i:Integer):Integer
  cnt:=0
  j=1..9
    s[i,j]=5  T / F
    cnt:=cnt+1 | -
  cntfive:=cnt
  Return value
```

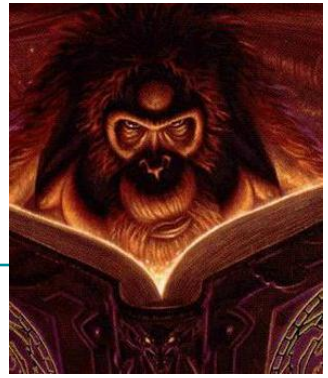
Example – excellent



At the Unseen University, at the end of the year, we know the grade of "n" wizards in "m" subjects in a table. The Librarian would like to reward the excellent students with a banana. How many bananas should he buy?



Example – excellent



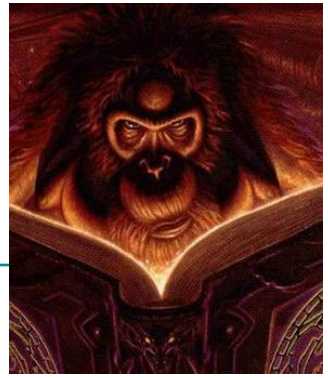
Task: how many rows contains only 5's in a matrix of integers?

	1	2	3	m=4
1	5	5	4	3
2	5	5	5	5
3	5	3	2	4
4	5	5	5	5
n=5	5	3	4	5

Idea:

- how many rows do we have …
- the answer is a number
- based on the **counting pattern**

Example – excellent



Task: how many rows contains only 5's in a matrix of integers?

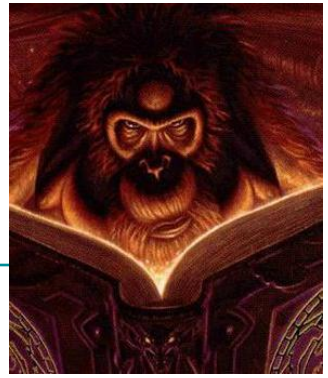
Specification:

```
In: n∈N, m∈N, grades∈N[1..n,1..m]
Out: cnt∈N
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: cnt=COUNT(row=1..n, onlyfive(row))
```

The onlyfive function contains the "subtask"

	1	2	3	m=4
1	5	5	4	3
2	5	5	5	5
3	5	3	2	4
4	5	5	5	5
n=5	5	3	4	5

Example – excellent



Task: how many rows contains only 5's in a matrix of integers?

row that contains only 5's:

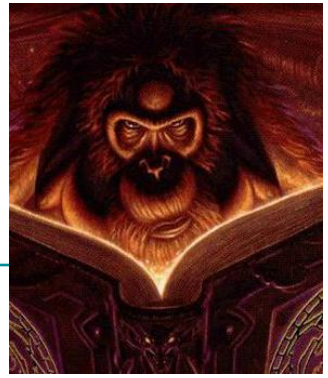
- focus on one row …
- the answer is a true/false
- based on the **decision(all) pattern**

Specification (onlyfive):

```
In: n∈N, m∈N, grades∈N[1..n,1..m],row∈N
Out: all∈L
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: (all=onlyfive(row))=
      all=FORALL(col=1..m, grades[row,col]=5))
```

	1	2	3	m=4
1	5	5	4	3
2	5	5	5	5
3	5	3	2	4
4	5	5	5	5
n=5	5	3	4	5

Example – excellent



Task: how many rows contains only 5's in a matrix of integers?

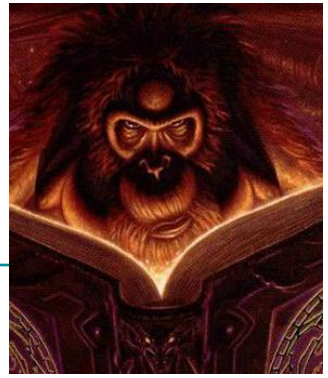
row that contains only 5's:

- focus on one row …
- the answer is a true/false
- based on the **decision(all) pattern**

Specification (onlyfive):

```
In: n∈N, m∈N, grades∈N[1..n,1..m],row∈N
Out: onlyfive(row)∈L
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: onlyfive(row)=FORALL(col=1..m,
grades[row,col]=5))
```

Example – excellent



Task: how many rows contains only 5's in a matrix of integers?

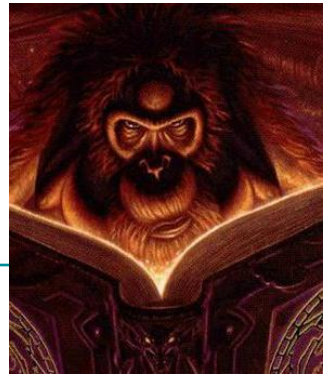
row that contains only 5's:

- focus on one row …
- the answer is a true/false
- based on the **decision(all) pattern**

Specification (onlyfive):

```
In: n∈N, m∈N, grades∈N[1..n,1..m],row∈N
Out: onlyfive(row)∈L
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: (all=onlyfive(row))=
      (all=FORALL(col=1..m, grades[row,col]=5))
```

Example – excellent



Task: how many rows contains only 5's in a matrix integers?

Specification:

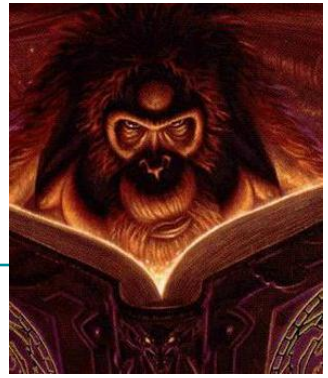
```
In: n∈N, m∈N, grades∈N[1..n,1..m]
Out: cnt∈N
Fn: onlyfive: N->L,
    onlyfive(row)= all∈L:
                FORALL(col=1..m, grades[row,col]=5))
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: cnt=COUNT(row=1..n, onlyfive(row))
```

grades, n, m are global

"all" for which the part after is true

	1	2	3	m=4
1	5	5	4	3
2	5	5	5	5
3	5	3	2	4
4	5	5	5	5
n=5	5	3	4	5

Example – excellent



counting pattern:

```
In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e, A(i))
```

decision all pattern:

```
In: b∈Z, e∈Z
Out: all∈L
Pre: -
Post: all=ALL(i=b..e, A(i))
```

specific task:

```
In: n∈N, m∈N, grades∈N[1..n,1..m]
Out: cnt∈N
Fn: onlyfive: N->L,
    onlyfive(row)= all∈L:
     FORALL(col=1..m,grades[row,col]=5))
Pre: ...
Post: cnt=COUNT(row=1..n, onlyfive(row))
```

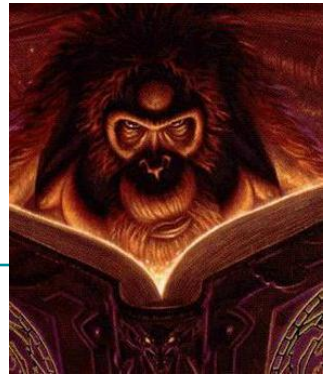
counting

i	~	row
b..e	~	1..n
A(i)	~	onlyfive(row)

decision all

i	~	col
b..e	~	1..m
A(i)	~	grades[row,col]=5

Example – excellent



Task: how many rows contains only 5's in a matrix of integers?

Specification:

```
In: n∈N, m∈N, grades∈N[1..n,1..m]
Out: cnt∈N
Fn: onlyfive: N->L,
    onlyfive(row)= all∈L:
                FORALL(col=1..m, grades[row,col]=5))
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: cnt=COUNT(row=1..n, onlyfive(row))
```

This redundant information will be omitted in the following. Assume that it specifies the output of the pattern! It will not appear in the reduction.

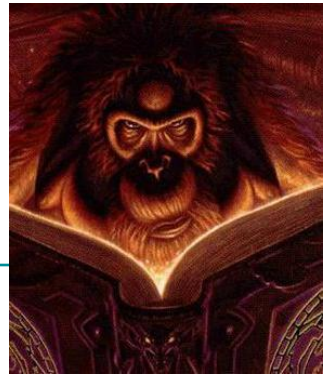
counting

```
i       ~ row
b..e    ~ 1..n
A(i)    ~ onlyfive(row)
```

decision all

```
i     ~ col
b..e  ~ 1..m
A(i)  ~ grades[row,col]=5
```

Example – excellent



Task: how many rows contains only 5's in a matrix of integers?

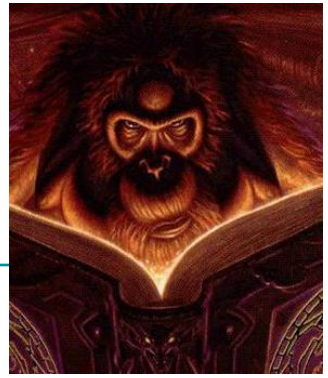
Specification:

```
In: n∈N, m∈N, grades∈N[1..n,1..m]
Out: cnt∈N
Fn: onlyfive: N->L,
    onlyfive(row)= FORALL(col=1..m, grades[row,col]=5))
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: cnt=COUNT(row=1..n, onlyfive(row))
```

counting		
i	~	row
b..e	~	1..n
A(i)	~	onlyfive(row)

decision all		
i	~	col
b..e	~	1..m
A(i)	~	grades[row,col]=5

Example – excellent



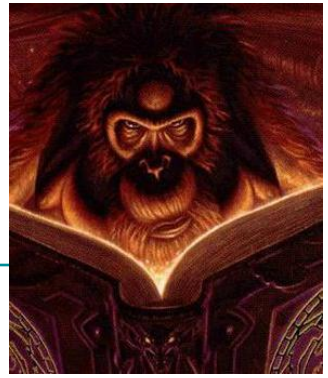
Comment: the patterns can be combined

Specification:

```
In: n∈N, m∈N, grades∈N[1..n,1..m]
Out: cnt∈N
Fn: onlyfive: N->L,
    onlyfive(row)= FORALL(col=1..m, grades[row,col]=5))
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: cnt=COUNT(row=1..n, onlyfive(row))
```

```
In: n∈N, m∈N, grades∈N[1..n,1..m]
Out: cnt∈N
Pre: ∀r∈[1..n]:(∀c∈[1..m]:( 1<=grades[r,c]<=5)
Post: cnt=COUNT(row=1..n, FORALL(col=1..m, grades[row,col]=5))
```

Example – excellent



Patterns

```
cnt:=0
i=b..e
    A(i)
T       F
    cnt:=cnt+1   -
```

counting

i	~	row
b..e	~	1..n
A(i)	~	onlyfive(row)

```
cnt:=0
row=1..n
    onlyfive(row)
T                F
    cnt:=cnt+1   -
```

2 separated tasks.

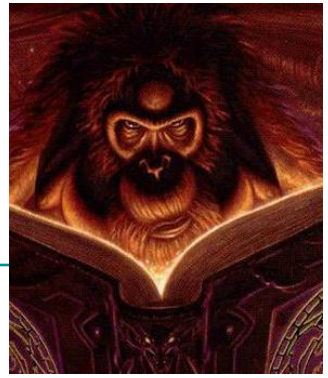
```
i:=b
i≤e and A(i)
    i:=i+1
all:=(i>e)
```

```
col:=1
col≤m and grades[col,row]=5
    col:=col+1
all:=(col>m)
```

decision all

i	~	col
b..e	~	1..m
A(i)	~	grades[row,col]=5

Example – excellent



Algorithm

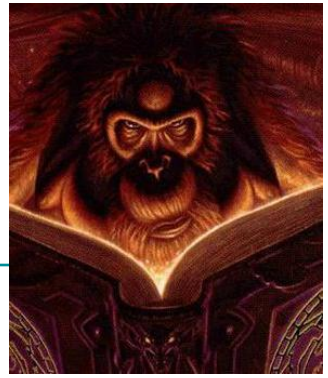
connecting:

1. **calling a function**: the nested algorithm will be a function and the "outer" algorithm call it
2. **integration**: copy the nested algorithm

```
cnt:=0
row=1..n
  T onlyfive(row) F
  cnt:=cnt+1 | -
```

```
col:=1
col ≤ m and grades[col,row]=5
  col:=col+1
all:=(col>m)
```

Example – excellent



connecting:

1. **calling a function**: the nested algorithm will be a function, and the "outer" algorithm call it
2. **integration**: copy the nested algorithm

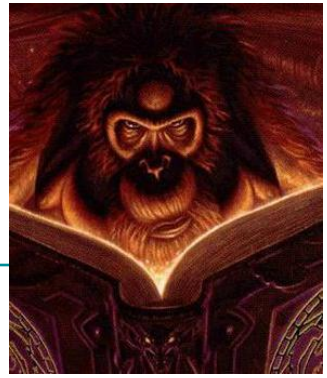
FOLLOW THIS!

Algorithm

```
cnt:=0
row=1..n
  T onlyfive(row) F
  cnt:=cnt+1 | -
```

```
Function
onlyfive(row:Integer):Boolean
  col:=1
  col≤m and grades[col,row]=5
    col:=col+1
  all:=(col>m)
  onlyfive:=all
  Return value
```

Example – excellent, the code



Conclusions

1. In a matrix, for each dimension can be (at least) one pattern
2. Try to break it down into subproblems (from the top to bottom)
3. Implement as function (algorithm, code)

Example
max. column

	1	2	3	4	n=5
1	1	2	4	3	1
2	2	6	4	2	1
3	5	3	2	1	0
4	2	8	2	8	2
n=5	5	3	2	3	0

Task: Find the column of the quadratic matrix t in which the **sum of the** elements in the principal diagonal and above is the **largest**.

	1	2	3	4	n=5
1	1	2	4	3	1
2	2	6	4	2	1
3	5	3	2	1	0
4	2	8	2	8	2
n=5	5	3	2	3	0

Example
max. column

	1	2	3	4	n=5
1	1	2	4	3	1
2	2	6	4	2	1
3	5	3	2	1	0
4	2	8	2	8	2
n=5	5	3	2	3	0

Task: Find the column of the quadratic matrix t in which the **sum of the** elements in the principal diagonal and above is the **largest**.

Steps:

1. Find a column based on the sum of elements above the target diagonal
2. Where this value is the largest -> column searched for
3. Summation in a Maximum selection

	1	2	3	4	n=5
s	1	8	10	14	4

	1	2		4	n=5
1	1	2	4	3	1
2	2	6	4	2	1
3	5	3	2	1	0
4	2	8	2	8	2
n=5	5	3	2	3	0

maxc=4

Example
max. column

	1	2	3	4	n=5
1	1	2	4	3	1
2	2	6	4	2	1
3	5	3	2	1	0
4	2	8	2	8	2
n=5	5	3	2	3	0

Task: Find the column of the quadratic matrix t in which the **sum of the** elements in the principal diagonal and above is the **largest**.

Specification:

```
In: n∈N, t∈Z[1..n,1..n]
Out: maxc∈N
Fn: sum:N->Z, sum(c)=SUM(r=1..c,t[r,c])
Pre: -
Post: (maxc,)=MAX(c=1..n,sum(c))
```

	1	2	3	4	n=5
s	1	8	10	14	4

maxc=4

	1	2	3	4	n=5
1	1	2	4	3	1
2	2	6	4	2	1
3	5	3	2	1	0
4	2	8	2	8	2
n=5	5	3	2	3	0

Example
max. column

	1	2	3	4	n=5
1	1	2	4	3	1
2	2	6	4	2	1
3	5	3	2	1	0
4	2	8	2	8	2
n=5	5	3	2	3	0

Maximum selection:

```
In: b∈Z, e∈Z
Out: maxind∈Z, maxval∈H
Pre: e<=u
Post: (maxind,maxval)=
            MAX(i=b..e,f(i))
```

Summation:

```
In: b∈Z, u∈Z
Out: s∈N
Pre: -
Post: db=SUM(i=b..e, f(i))
```

Specification:

```
In: n∈N, t∈Z[1..n,1..n]
Out: maxc∈N
Fn: sum:N->Z,
    sum(c)=SUM(r=1..c,t[r,c])
Pre: -
Post: (maxc,)=MAX(c=1..n,sum(c))
```

maximum selection		
i	~	c
maxind	~	maxc
b..e	~	1..n
f(i)	~	sum(i)

summation		
i	~	r
b..e	~	1..c
f(i)	~	t[r,c]

Example
max. column

	1	2	3	4	n=5
1	1	2	4	3	1
2	2	6	4	2	1
3	5	3	2	1	0
4	2	8	2	8	2
n=5	5	3	2	3	0

Maximum selection:

```
maxind:=b, maxval:=f(b)
i=b+1..e
  f(i)>f(maxind)
  T: maxind:=i
     maxval:=f(i)
  F:
```

maximum selection

i	~	c
maxind	~	maxc
b..e	~	1..n
f(i)	~	sum(i)

```
maxc:=1
i=2..n
  sum(i)>sum(maxc)
  T: maxc:=i
  F: -
```

Summation:

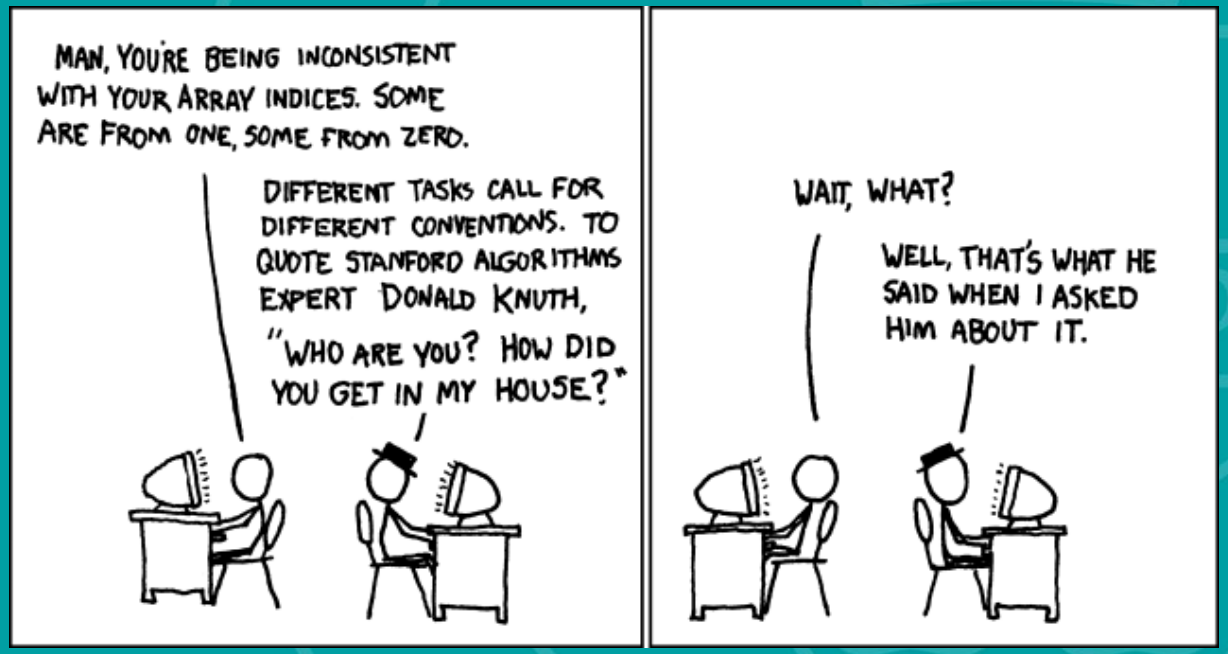
```
s:=0
i=b..e
  s:=s+f(i)
```

summation

i	~	r
b..e	~	1..c
f(i)	~	t[r,c]

```
Function
sum(c:Integer):Integer
  s:=0
  r=1..n
    s:=s+t[r,c]
  sum:=s
  Return value
```

ELTE
FACULTY OF
INFORMATICS



Thank you for your attention!